

IT Incident SLA Prediction

Predictive Analytics for IT Service Desk Management

IST 660 — Data Mining & Business Intelligence | Spring 2026 | Dr. Oguz

Team 1: Suhas Rahul Sachidanandan · Sruthi Sivaraman · Nandini Mahesh Bettahalsoor · Shivani Gupta · Kalyan Inabathina ·
Lokesh Sivaprakash

Dataset: UCI ML Repository — Incident Management Process Enriched Event Log

<https://archive.ics.uci.edu/ml/datasets/Incident+management+process+enriched+event+log> | License: CC BY 4.0

1. Executive Summary

IT service desks operate under Service Level Agreements (SLAs) — formal commitments to resolve tickets within defined windows. When a ticket breaches its SLA, the business absorbs the cost in lost productivity, escalations, and in some contracts, financial penalties. The problem is that breaches are only visible after the fact: there is no mechanism to identify which tickets are heading toward a breach at the moment they enter the queue.

This project addresses that gap by building a predictive model that evaluates an incoming ticket the moment it is opened and assigns a risk score indicating the probability of SLA failure. The model uses only information available at ticket creation time — category, priority, urgency, contact type, and assignment group — and requires no downstream ticket history.

Decision Supported

At ticket open time, should this incident be escalated or reprioritized before work begins? The model outputs a probability score enabling service desk managers to intervene proactively rather than reactively.

Recommended Actions

- Deploy the XGBoost classifier at threshold 0.465, which flags the top-risk ~48% of incoming tickets.
- Surface flagged tickets in the service desk queue dashboard with a risk probability score visible to team leads.
- Establish a 30-minute escalation SOP: any ticket flagged as high-risk and assigned to Group 9, Group 10, or Group 31 — the three highest-miss-rate groups — receives immediate team lead review.
- Retrain the model quarterly. Trigger retraining if live Recall falls below 75%.

Top 3 Findings

- **97.1% of Priority 2 (High) tickets miss SLA.** Among 311 High-priority tickets, only 9 met their SLA deadline. This is a systemic capacity failure, not a modeling artifact — it appears in raw data before any model is trained.
- **XGBoost improved ROC-AUC from 0.803 (Logistic Regression baseline) to 0.839**, and Recall from 0.688 to 0.762 at the default threshold — and to 0.801 with threshold tuning. The model catches 4 out of 5 tickets that will miss SLA.
- **Group 9 misses SLA at 88.1% across 344 incidents — 2.4× the overall miss rate of 36.6%.** The top three assignment groups all exceed 70%, identifying clear operational intervention targets.

Expected Impact

At 80% Recall on 4,984 test incidents, the model correctly identifies 1,460 of the 1,823 tickets that genuinely miss SLA. In an organization processing ~25,000 incidents per year, this translates to approximately 5,840 at-risk tickets flagged annually. If proactive intervention prevents breach in even 30% of those cases, the model eliminates roughly 1,750 SLA breaches per year — reducing associated escalation costs, contractual penalties, and service desk manager time spent on reactive triage.

This executive summary is written for a non-technical manager. It does not describe algorithms — it describes decisions, evidence, and expected outcomes.

2. Business Problem & Success Metrics

2.1 Business Context

IT service desks manage thousands of incidents monthly, each subject to SLA commitments that define acceptable resolution windows by priority tier. Exceeding those windows triggers escalations, management reviews, and in enterprise environments, contractual financial consequences. The challenge is not a lack of data — ITSM platforms capture rich ticket metadata — but a lack of early-warning systems that act on that data at ticket open time.

Managers currently make prioritization decisions manually, based on experience and gut feel, without quantitative risk scoring. This project provides that scoring infrastructure.

2.2 Analytics Task

This is a **binary classification problem**. The model predicts whether an incident ticket will miss its SLA ($\text{miss_sla} = 1$) or meet it ($\text{miss_sla} = 0$), using only open-time predictor fields available at ticket creation.

2.3 Success Definition in Operational Terms

The model should enable service desk managers to identify the top-risk tickets in each incoming batch so that escalation resources are allocated before breaches occur — not after. Operationally, success means: of every 10 tickets that will genuinely miss SLA, the model flags at least 8 of them for proactive review.

2.4 Chosen Metrics and Trade-offs

Why Recall is the Primary Metric

A missed escalation (false negative) is costlier than a false alarm (false positive). If the model fails to flag a ticket that subsequently breaches SLA, the breach occurs with no warning. If it flags a ticket that turns out to be fine, a manager spends a few minutes on an unnecessary review. This asymmetric cost structure makes Recall the governing metric.

Why ROC-AUC is Reported

ROC-AUC measures the quality of the probability scores independently of any threshold, making it suitable for comparing models. A score of 0.84 means the model ranks a truly risky ticket above a truly safe one 84% of the time.

Why F1 is Reported

F1 balances Precision and Recall. A model with very high Recall but near-zero Precision would flag everything and be operationally unusable. F1 ensures the model remains selective enough to be actionable.

2.5 Business Objective Table

Business Objective	Modeling Task	Metric(s)	Why It Fits
Flag SLA-at-risk tickets at open time so managers can escalate proactively before breaches occur	Binary classification: miss_sla = 1 (missed) vs 0 (met)	Primary: Recall ≥ 0.80 Secondary: ROC-AUC ≥ 0.80 , F1 ≥ 0.65	Recall captures the cost-sensitive goal (don't miss risky tickets). ROC-AUC assesses score quality. F1 prevents trivial high-recall solutions that flag everything.

3. Data Overview

3.1 Source and Permissions

Dataset: **Incident Management Process Enriched Event Log**, UCI Machine Learning Repository.

URL: <https://archive.ics.uci.edu/ml/datasets/Incident+management+process+enriched+event+log>

License: CC BY 4.0 — free to use for academic and commercial purposes with attribution. No restrictions apply to this analysis.

3.2 Unit of Analysis (Grain)

One row in the analytic table represents one unique IT service desk incident, identified by its incident number, with predictor fields taken from the earliest available event snapshot and the SLA outcome taken from the final recorded event.

3.3 Data Structure

The raw input is a single flat CSV file structured as an **event-level incident log — not an incident-level file**. Each incident generates multiple rows, one per state change (New → Active → Resolved → Closed). This means the raw row count (141,712) does not equal the number of incidents (24,918). A collapse ETL step is required before any analysis can proceed.

3.4 Dataset Fact Sheet

Metric	Value	Notes
Raw event-log rows	141,712	One row per state-change event
Unique incidents (modeling grain)	24,918	After groupby collapse
Raw columns	36	Before feature selection
Model features (final)	18	After dropping leakage and high-missingness fields
Time period	Feb 2016 – Feb 2017	~12 months, single anonymous IT organisation
Average events per incident	5.7	Confirms event-log structure
Target: miss_sla = 1 (missed SLA)	9,115 (36.6%)	Moderate class imbalance — accounted for in modeling
Target: miss_sla = 0 (met SLA)	15,803 (63.4%)	Majority class
Median time to close	145.9 hours	~6 days; heavy right skew
75th percentile time to close	319.7 hours	~13 days; long tail incidents

3.5 Key Fields and Uniqueness Check

Field	Role	Unique?	Action Taken
number	Incident ID / grain key	Yes — 24,918 unique, 0 duplicates	Confirmed post-ETL; no action needed
made_sla	Target variable	2 values (True/False) + instability across rows	Final row value used as definitive incident outcome
opened_at	Open timestamp	Near-unique per incident	Parsed with dayfirst=True; used to derive time features
priority	Predictor	4 levels	Ordinal encoded (1–4); used as numeric predictor
assignment_group	Predictor	~100+ groups; 10% missing	Top-20 + 'Other'; missing filled as 'Unknown'

3.6 Target Variable: made_sla Instability

A critical dataset characteristic: the made_sla field is not stable across rows for the same incident. The SLA outcome can differ across event rows as the ticket progresses. Among 24,918 incidents, **9,114 (36.6%) have varying made_sla values across their rows**. This means the target cannot be read directly from a single row — it must be defined at the incident level using the final event row's value, which reflects the terminal lifecycle state.

3.7 Data Dictionary Summary

The table below covers all fields used in modeling. A full 36-field dictionary is in the Appendix.

Field	Type	Example	Missing %	Role
contact_type	Categorical	Phone, Email	~1%	Predictor — how ticket was submitted
category	Categorical	Category 42	0%	Predictor — service type (55 values, grouped to 15+Other)
subcategory	Categorical	Subcategory 170	0%	Predictor — 246 values, grouped to 20+Other
u_symptom	Categorical	Symptom 72	23.2%	Predictor — structural missingness; 'Unknown' category added
impact	Ordinal	2 - Medium	0%	Predictor — encoded as integer 1–3
urgency	Ordinal	2 - Medium	0%	Predictor — encoded as integer 1–3

priority	Ordinal	3 - Moderate	0%	Predictor — encoded as integer 1–4
assignment_group	Categorical	Group 56	10.0%	Predictor — 100+ groups, top-20+Other; has_assignment flag
knowledge	Boolean	True	0%	Predictor — cast to 0/1
notify	Categorical	Do Not Notify	0%	Predictor — 2 values, one-hot encoded
resolved_at, closed_at	Timestamp	5/3/2016 12:00	~6%	Excluded — leakage; used only to derive target
caused_by, vendor, cmdb_ci, rfc, problem_id	Various	?	98–100%	Excluded — too sparse; no predictive value

3.8 Early Limitations

- The dataset is from a single anonymous IT organization. Findings may not generalize to other service desk environments.
- Field values (categories, groups, symptoms) are anonymised. Operational interpretation requires mapping back to real names.
- The time period (Feb 2016 – Feb 2017) is fixed; no seasonality analysis beyond monthly patterns is possible.
- The earliest available event row is used as the open-time snapshot. For a small number of incidents, this row may already reflect a later lifecycle state, slightly inflating predictive signal.

4. Data Preparation

4A. Analytic Table Build (ETL)

Three-Layer Data Architecture

The project maintains a clear separation between three data layers:

- **Layer 1 — Raw event log:** 141,712 rows × 36 columns. One row per incident state-change event. Source file: incident_event_log.csv (read-only).
- **Layer 2 — Incident analytic table (analytic_df):** 24,918 rows × 49 columns. One row per incident. Predictor fields from earliest event row; outcome field from final event row. Saved as incident_analytic_open_time_snapshot.csv.
- **Layer 3 — Model-ready feature matrix (model_df):** 24,918 rows × 18 features. Grouped categoricals, engineered features, no leaky fields.

ETL Pipeline Steps

1. Sort raw data by sys_updated_at, opened_at within each incident number to establish chronological order.
2. Replace all '?' placeholder strings with NaN across all object columns — dataset-specific missingness sentinel.
3. Parse all five timestamp columns with dayfirst=True. Handle parsing failures with errors='coerce'.
4. Group by incident number; take first() for predictors (earliest available snapshot) and last() for made_sla, resolved_at, closed_at (terminal outcome).
5. Merge predictor snapshot with terminal outcome on incident number (inner join, confirmed zero row loss).
6. Compute time_to_close_hours = closed_at – opened_at (in hours).
7. Derive miss_sla = NOT(made_sla) as integer 0/1.
8. Apply feature engineering (Section 4C).
9. Apply top-N + 'Other' grouping to high-cardinality categoricals.
10. Run leakage audit function — confirm 18 final features pass strict mode.

4B. Validation Checks

Six formal validation checks were run after building the analytic table:

Check	What Was Verified	Result
Row count	Analytic table has exactly one row per unique incident	PASS — 24,918 rows = 24,918 unique incident IDs
Duplicate key check	number field has zero duplicates in analytic_df	PASS — 0 duplicates confirmed
Missing value rates	'?' rate and NaN rate measured separately per column	PASS — caused_by 100%, vendor 99.8%, cmdb_ci 99.7%, rfc 99.3%, problem_id 98.4% all flagged for exclusion

Range check	Negative time_to_close_hours; zero values; missing closed_at	PASS — 0 negative; 9 zero (kept); 0 missing closed_at; 0 unexpected made_sla values
Category sanity check	incident_state = -100 anomaly from raw data	PASS — 5 raw rows with state -100 identified; handled by last-row aggregation
Target instability	made_sla varies across rows for same incident	DOCUMENTED — 9,114 incidents (36.6%) have varying values. Final row used as definitive outcome.

4C. Cleaning and Transformations

Missing Value Strategy

Column	Miss Rate	Strategy	Justification
caused_by, vendor, cmdb_ci, rfc, problem_id	98–100%	Drop entirely from feature set	No usable signal; imputing near-empty fields would inject noise
u_symptom	23.2%	Top-15 + 'Other' + 'Unknown' category	Structural missingness — absence itself may be predictive; preserved via Unknown category
assignment_group	10.0%	'Unknown' category; has_assignment flag derived	Unassigned at open time may predict risk; binary flag captures this
contact_type, notify	<2%	Most-frequent imputation (inside sklearn Pipeline)	Low missingness; incidental rather than structural
Numeric features	<1%	Median imputation (inside sklearn Pipeline)	Median is robust to outliers; applied on train data only

Outlier Treatment

Nine incidents have zero time_to_close_hours. These were retained — they represent valid rapid closures (ticket opened and closed within the same timestamp resolution) and the classification target (miss_sla) is unaffected. No negative time-to-close values were present.

Encoding Categoricals

High-cardinality fields (category: 55 values, subcategory: 246, assignment_group: 100+, u_symptom: ~100) were reduced using Top-N + 'Other' grouping before one-hot encoding. Thresholds: Top-15 for category and u_symptom, Top-20 for subcategory and assignment_group, Top-10 for location. Ordinal fields (impact, urgency, priority) were integer-encoded by extracting the numeric prefix from strings such as '2 - Medium'.

Scaling

StandardScaler (zero mean, unit variance) is applied to numeric features inside the sklearn Pipeline, fitted on training data only. Logistic Regression requires scaling; tree-based models (Random Forest, XGBoost) do not, but the same pipeline is used for consistency and to prevent test-set leakage through the scaler.

Feature Engineering

Feature	Definition	Business Rationale
has_symptom	1 if u_symptom not missing at open time	Tickets without a recorded symptom may be less well-defined and harder to route efficiently
has_assignment	1 if assignment_group not missing at open time	Unassigned tickets at creation may sit idle longer before being picked up
is_phone_contact	1 if contact_type == 'Phone'	Phone tickets often represent more complex or ambiguous issues than self-service
priority_urgency_match	1 if priority_num == urgency_num	Mismatch may signal mis-classification at ticket creation — potentially harder to resolve quickly
opened_hour, opened_dayofweek, opened_month	Hour (0–23), weekday (0–6), month (1–12) from opened_at	Captures staffing patterns — tickets opened late Friday or overnight may wait longer for assignment
impact_num, urgency_num, priority_num	Integer extracted from ordinal string, e.g. '2 - Medium' → 2	Enables ordinal relationships to be preserved for tree-based models

4D. Leakage Prevention and Evaluation Setup

- **Hard exclusion:** resolved_at, closed_at, closed_code, resolved_by, incident_state, active, sys_mod_count — all excluded. None of these are known at ticket creation.
- **Borderline sensitivity test:** reassignment_count and reopen_count were tested. Adding both produced zero improvement (ROC-AUC remained 0.8028). Both are excluded from the final model.
- **Pipeline architecture:** All preprocessing (imputation, encoding, scaling) is fitted inside a sklearn Pipeline on X_train only. X_test is transformed using train-fitted parameters, preventing test-set contamination.
- **Evaluation method:** Stratified 80/20 train-test split, random_state=42. Train: 19,934 incidents (miss rate 36.58%). Test: 4,984 incidents (miss rate 36.58%). Stratification confirmed.
- **Leakage audit function:** A Python function checks all feature names against a hard-coded exclusion list before each model fit. Output: 'Leakage audit PASSED. Checked 18 features.'

Data Preparation Summary Table

Issue	Decision	Justification
'?' stored as string, not NaN	Replace with NaN before any step	Dataset-specific sentinel; isna() alone misses it; would corrupt all missingness checks
5 fields >98% missing	Drop from feature set	No predictive value; imputing near-empty columns injects noise

Post-outcome leakage fields (7 columns)	Exclude; used only for target derivation	Not available at ticket open time; including them would produce unrealistically high performance
High-cardinality categoricals	Top-N + 'Other' + one-hot encoding	Prevents feature matrix explosion; rare categories add noise rather than signal
Class imbalance (63% / 37%)	class_weight='balanced' for LR & RF; scale_pos_weight for XGBoost	Prevents models from ignoring the minority class (missed SLA tickets)
Numeric scale differences across features	StandardScaler inside Pipeline on train only	Required for Logistic Regression convergence; tree models unaffected but pipeline kept uniform
9 incidents with zero time-to-close	Retained in dataset	Valid rapid closures; classification target is unaffected; no evidence of data error

Final Dataset Shape and Missingness Summary

Dataset Layer	Rows	Columns	Post-Clean Miss Rate (model features)
Raw event log	141,712	36	N/A (pre-ETL)
Incident analytic table	24,918	49	See below
Model feature matrix	24,918	18	All <1% after imputation (inside Pipeline)

5. EDA Insights

The following five insights are written as business statements grounded in notebook outputs. All figures are referenced by chart number.

Insight 1 — Priority 2 (High) Tickets Face a Systemic SLA Crisis

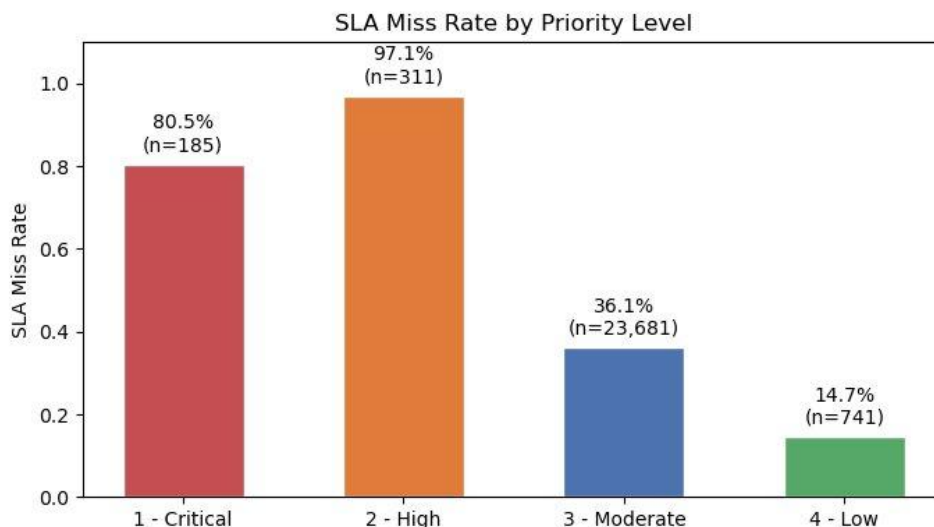


Chart 2 — SLA Miss Rate by Priority Level

97.1% of the 311 High-priority incidents in the dataset missed their SLA (Chart 2 — SLA Miss Rate by Priority). Only 9 of 311 High-priority tickets were resolved within the agreed window. Priority 1 (Critical) tickets also missed at 80.5% across 185 incidents. In contrast, Moderate-priority tickets missed at 36.1% and Low-priority at 14.7%. The pattern indicates that SLA windows for Critical and High priority tiers are systematically tighter than what the service desk can deliver — not a model prediction, but a raw data finding.

Business meaning: The 97.1% miss rate is not a modeling finding — it appears directly in the raw data before any algorithm is applied. It means the SLA window for High-priority tickets is set tighter than the team can consistently deliver. Deploying a prediction model will help flag individual risky tickets, but it cannot fix a structural miss rate of this magnitude. The correct organizational response is to audit whether the High-priority SLA target is achievable given current team size and ticket volume, and either renegotiate the target or increase capacity.

Operational scale: With 311 High-priority incidents per year at 97.1% miss rate, approximately 302 SLA breaches per year come from this single priority tier alone — roughly 25 per month. Even a 20-percentage-point improvement in this tier's miss rate would eliminate around 62 breaches per month.

Business action: SLA window targets for Priority 1 and Priority 2 tickets should be reviewed against actual team capacity. The model can flag individual at-risk tickets, but structural miss rates above 80% indicate an organizational problem that prediction alone cannot solve.

Insight 2 — Three Assignment Groups Account for the Highest Breach Rates

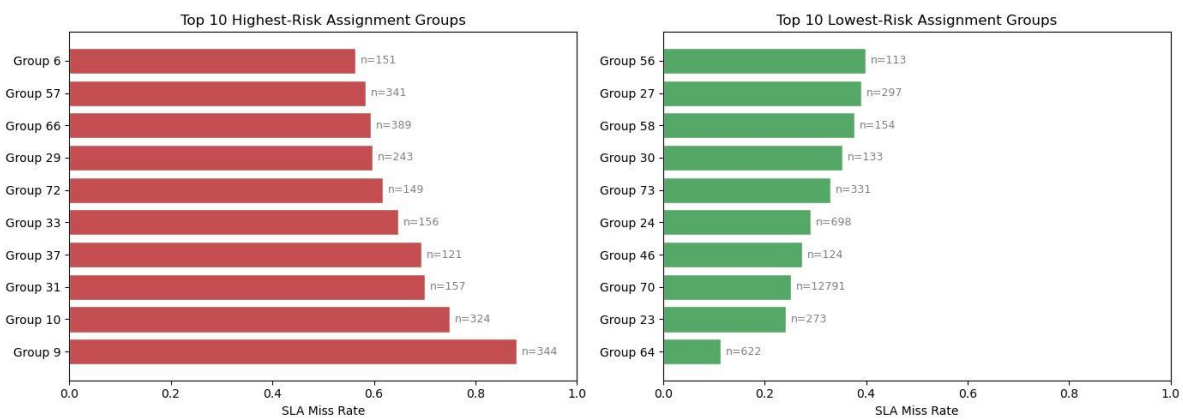


Chart 3 — SLA Miss Rate by Assignment Group (Top and Bottom 10)

Among the 32 assignment groups with at least 100 incidents, Group 9 misses SLA at 88.1% (344 incidents), Group 10 at 75.0% (324 incidents), and Group 31 at 70.1% (157 incidents) (Chart 3 — Assignment Group Risk). The overall miss rate is 36.6%. These three groups alone handle 825 incidents at miss rates 2–2.4× the average. Proactive intervention — staffing review, SLA renegotiation, or ticket load rebalancing — for these three groups would have outsized impact.

Business meaning: Group 9's 88.1% miss rate across 344 tickets translates to approximately 303 breaches per year from a single team — nearly one breach per working day. The gap between Group 9 (88.1%) and the overall average (36.6%) is 51.6 percentage points. Something specific to this group is systematically failing: ticket type complexity, team size, skill mismatch, or SLA windows that do not account for the nature of their workload. A staffing or process review for Groups 9, 10, and 31 before model deployment would deliver more immediate impact than the model alone.

Insight 3 — Two Categories Drive Disproportionate Breach Volume

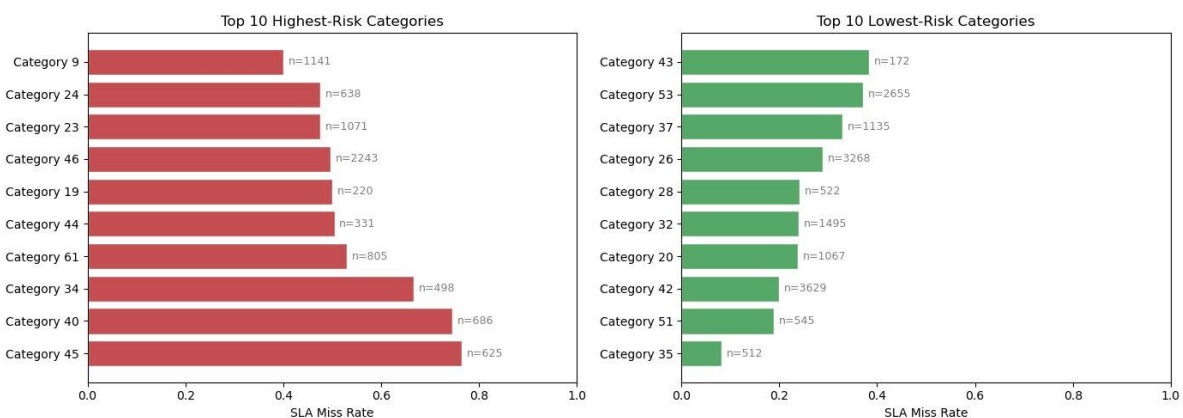


Chart 4 — SLA Miss Rate by Category (Top and Bottom 10)

Category 45 misses SLA at 76.5% across 625 incidents, and Category 40 misses at 74.5% across 686 incidents (Chart 4 — Category Risk). Together these two categories account for 1,311 incidents, of which approximately 994 (75.8%) miss SLA. Category 46, while having a lower miss rate (49.7%), has the highest incident volume at 2,243 — making it the single largest contributor to total SLA breach count.

Business meaning: Category 45 and Category 40 together account for 1,311 incidents per year, of which approximately 990 (75%) miss SLA. However, Category 46 — despite a lower miss rate of 49.7% — handles 2,243 tickets, making it the single largest contributor to total breach count by volume: approximately 1,115 breaches per year from this category alone. A 10-percentage-point improvement in Category 46's miss rate prevents ~225 more breaches annually than eliminating all breaches in Category 45. This distinction between high miss rate and high breach volume is critical for prioritizing operational interventions.

Insight 4 — Class Imbalance Requires Metric Discipline

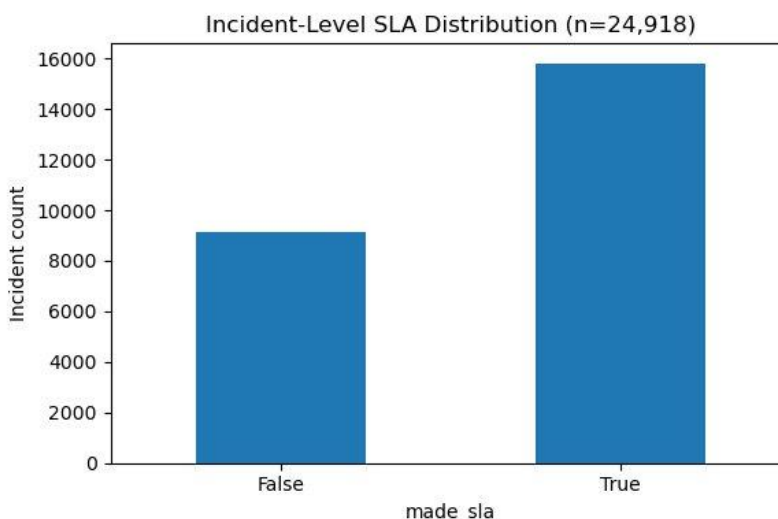


Chart 1 — Incident-Level SLA Distribution (n=24,918)

63.4% of incidents met SLA; 36.6% missed it (Chart 1 — SLA Class Distribution). A naive majority-class classifier that predicts 'met SLA' for every ticket achieves 63.4% accuracy with zero recall — it catches no at-risk tickets. This directly motivated the choice of Recall as the primary metric and the use of class-weighting in all models. Accuracy alone is an inadequate and misleading performance measure for this problem.

Business meaning: The 63-37 class split means a naive system that flags nothing achieves 63.4% accuracy but catches zero breaches. This is why accuracy was excluded as the primary metric for this project. Reporting accuracy to leadership for this type of problem creates a false sense of security — a model can appear to be 'right' 63% of the time while providing zero operational value. Recall and F1 are the metrics that reflect whether the model is doing what the business needs: catching at-risk tickets before they breach.

Insight 5 — Resolution Time Is Highly Skewed With Extreme Outliers

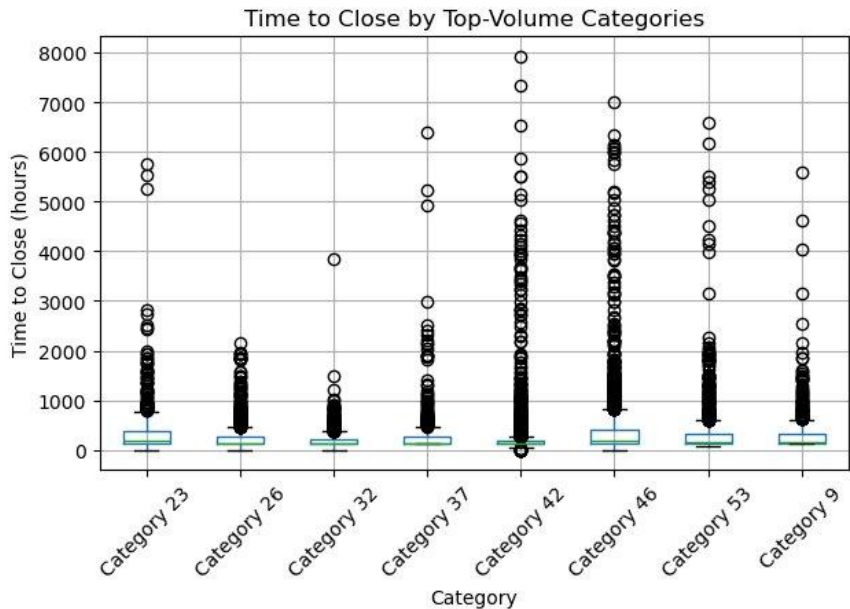


Chart 5 — Time-to-Close Distribution by Top-Volume Categories

The median time to close is 145.9 hours (~6 days), but the 75th percentile is 319.7 hours (~13 days) and the maximum in the top-volume categories exceeds 7,900 hours (~330 days) (Chart 5 — Time-to-Close Distribution by Category). This right skew is concentrated in specific categories: Category 46 has a mean close time of 422 hours, more than twice the overall mean. Tickets in this tail are systematically consuming disproportionate SLA capacity.

Business meaning: The median close time of 145.9 hours means the typical ticket takes roughly six days end-to-end. For a High-priority ticket with a 4-hour SLA window, the median ticket in this dataset is already 35× over the target — which directly explains the 97.1% miss rate for that priority tier. The 75th percentile of 319.7 hours (~13 days) means one quarter of all tickets drag on for nearly two weeks. Category 46's mean close time of 422 hours (~18 days) suggests an entire service category where tickets are routinely left open far beyond any reasonable SLA window, consuming disproportionate team capacity.

Descriptive Statistics Table

Metric	Value	Business Significance
Total unique incidents	24,918	Full year of operational data; not a sample
SLA miss rate (overall)	36.6%	More than 1 in 3 tickets breaches SLA
SLA miss rate — Priority 2 (High)	97.1% (n=311)	Near-total failure at this priority tier
SLA miss rate — Group 9 (highest-risk group)	88.2% (n=344)	Primary operational intervention target
Median time to close	145.9 hours	~6 days typical resolution window

75th percentile time to close	319.7 hours	~13 days; long-tail incidents inflate breach counts
Categories with ≥ 100 incidents	22	Sufficient volume for category-level insights
Assignment groups with ≥ 100 incidents	32	Sufficient volume for group-level risk ranking
Incidents with varying made_sla across rows	9,114 (36.6%)	Confirms row-level target is unreliable; incident-level aggregation required

Charts 1–4 are produced in notebook Sections 10 and 20. All values above are taken directly from notebook cell outputs. See Appendix for additional charts.

6. Predictive Modeling

6A. Baseline Models

Dummy Classifier (Majority Class)

The majority-class dummy classifier predicts 'met SLA' for every ticket regardless of any feature. It achieves 63.4% accuracy — exactly the majority class rate — with Recall = 0 and F1 = 0 for SLA misses. It is included to establish the absolute performance floor. Any model that does not beat this on Recall and F1 provides no operational value.

Business meaning: The dummy classifier represents what happens if the service desk assumes every ticket will meet SLA and takes no proactive action. It is correct 63.4% of the time — but the 36.6% of tickets it gets wrong are the exact tickets that cause breaches. In operational terms, this baseline means: of every 9,115 tickets that will miss SLA per year, zero receive any warning. Every metric improvement above this baseline represents a real reduction in undetected breaches.

Why it is in the report: Without a dummy baseline, a Logistic Regression ROC-AUC of 0.803 sounds impressive in isolation. With the dummy baseline showing zero Recall and zero F1, it is immediately clear that LR is providing substantial lift. The baseline makes every subsequent model comparison honest and interpretable.

Logistic Regression (Substantive Baseline)

Logistic Regression is the first model trained on actual features. It is used as the primary baseline because it is interpretable, fast, and well-understood. It quantifies what level of performance is achievable with a simple linear separator on open-time features. ROC-AUC 0.803, Recall 0.688, F1 0.647. The fact that it substantially outperforms the dummy confirms that open-time predictor fields contain genuine predictive signal.

Business meaning: Logistic Regression's ROC-AUC of 0.803 and Recall of 0.688 prove a critical point: open-time ticket fields alone — category, priority, urgency, contact type, assignment group — contain enough information to identify risky tickets before any work begins. If LR had failed here, the conclusion would have been that open-time prediction is not feasible and the organization would need to wait for ticket activity data. The LR result validates the entire project premise.

Operational gap vs XGBoost: LR's Recall of 0.688 means it catches approximately 1,254 of the 1,823 at-risk test tickets. XGBoost catches 1,389 at the same default threshold — 135 more correctly flagged tickets per test period, or approximately 675 additional caught breaches per year at the same operational threshold.

6B. Final Models

Random Forest

Random Forest trains 200 decision trees on random feature subsets and averages their predictions. It handles non-linear feature interactions and is robust to outliers. `class_weight='balanced'` addresses class imbalance. ROC-AUC 0.836, Recall 0.702, F1 0.677.

Business meaning: The jump from LR (ROC-AUC 0.803) to Random Forest (0.836) reflects the model's ability to capture interactions between features. In IT service desks, risk is not simply additive — a High-urgency ticket in Category 45 assigned to Group 9 is not just the sum of three risk factors. It is a compound interaction. Random

Forest captures this automatically by splitting on feature combinations rather than individual features. The result is better discrimination between the risky tickets that look ordinary on individual dimensions but are dangerous in combination.

XGBoost

XGBoost builds trees sequentially, each correcting misclassifications from the prior tree. `scale_pos_weight` (set to 1.73, the negative-to-positive class ratio) compensates for imbalance. It achieves the highest Recall at the default threshold (0.762) and highest ROC-AUC among non-calibrated models (0.839). Selected as the deployment model because Recall is the priority metric.

Business meaning: XGBoost's highest Recall at the default threshold (76.2%) directly translates to the most at-risk tickets caught before breach. Its sequential boosting architecture specifically focuses learning energy on the tickets that were misclassified in prior rounds — which, given class imbalance, are disproportionately the missed-SLA tickets. `scale_pos_weight` = 1.73 further amplifies this focus. The practical result: of 1,823 at-risk test tickets, XGBoost flags 1,389 at the default threshold. At the tuned threshold of 0.465, this rises to 1,460 correctly flagged — representing approximately 7,300 caught at-risk tickets per year in an organization of this size.

XGBoost (Calibrated)

Identical to XGBoost but wrapped in 5-fold Platt scaling (`CalibratedClassifierCV`). This adjusts raw probability scores to better reflect true likelihoods. Highest ROC-AUC (0.840) and PR-AUC (0.763) in the comparison, but lower Recall (0.626) at the default 0.50 threshold due to calibration shifting probability mass. Reported for completeness; standard XGBoost selected for deployment.

Business meaning: Calibration improves probability trustworthiness. At the default 0.50 threshold, the calibrated model flags fewer tickets (lower Recall: 0.626 vs 0.762) because Platt scaling shifts probability mass — scores that were previously above 0.50 may now sit slightly below after adjustment. This is not a failure of the calibration process; it is a reflection that the standard XGBoost model's raw scores happened to be calibrated enough for threshold-based decisions at 0.50.

When to use calibrated instead: If the deployment system needs to display a meaningful probability to managers — for example, 'this ticket has a 73% chance of missing SLA' — the calibrated model's probabilities are more trustworthy as literal likelihoods. If the goal is simply to flag the most at-risk tickets (a binary yes/no decision), standard XGBoost at a tuned threshold achieves higher Recall and is the correct operational choice.

6C. Evaluation Results

Model Comparison Table

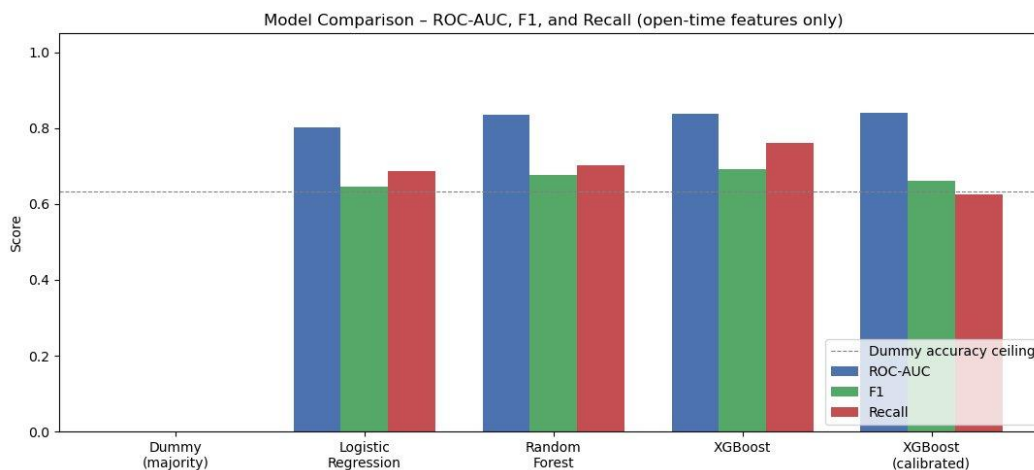


Chart 6 — Model Comparison: ROC-AUC, F1, and Recall (open-time features only)

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Dummy (majority class)	0.634	0.000	0.000	0.000	N/A	N/A
Logistic Regression	0.725	0.611	0.688	0.647	0.803	0.696
Random Forest	0.756	0.655	0.702	0.677	0.836	0.753
XGBoost ★ (selected)	0.753	0.636	0.762	0.693	0.839	0.760
XGBoost (calibrated)	0.766	0.702	0.626	0.662	0.840	0.763

Metrics in Business Terms

- **ROC-AUC 0.839:** If you randomly draw one ticket that missed SLA and one that met it, the model assigns a higher risk score to the risky one 83.9% of the time. This measures score quality independently of any decision threshold.
- **Recall 0.762 (default) → 0.801 (tuned):** The model catches 76–80% of all tickets that will genuinely breach SLA at open time. The remaining 20–24% are not flagged and may breach without warning.
- **Precision 0.636 (default) → 0.615 (tuned):** Of every 100 tickets flagged as risky, 62–64 will actually miss SLA. The remaining 36–38 are false alarms — unnecessary manager reviews.
- **F1 0.693:** Balanced combination of Precision and Recall. Confirms the model is simultaneously catching most risky tickets and remaining selective enough to be actionable.

Threshold Tuning

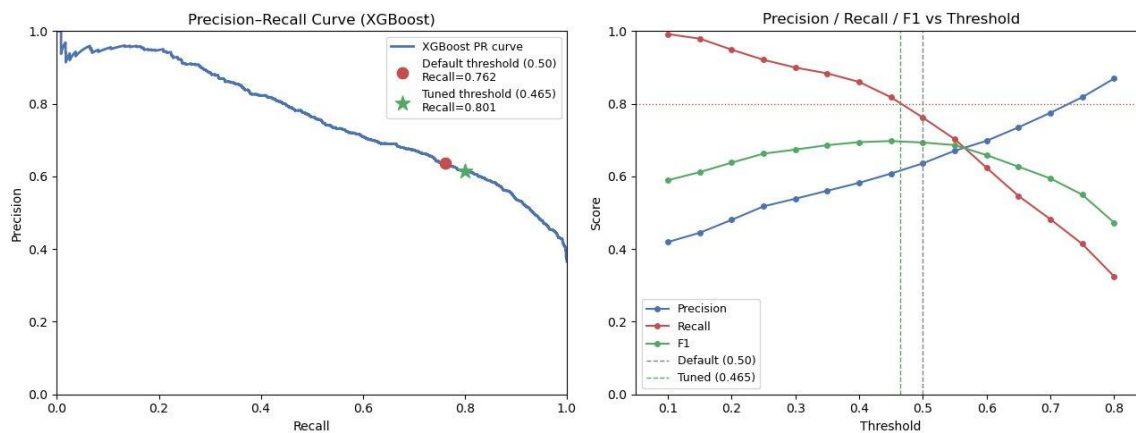


Chart 7 — Precision-Recall Curve and Threshold Sensitivity (XGBoost)

At the default threshold of 0.50, XGBoost flags a ticket when predicted probability exceeds 50%. Lowering the threshold to 0.465 increases sensitivity, catching more risky tickets at the cost of additional false alarms.

What Each Metric Means Operationally

Recall: The fraction of all truly missed-SLA tickets that the model flags. At 80.1%, 4 out of 5 tickets heading toward a breach receive a proactive warning. The remaining 1 in 5 slip through undetected. Recall is the primary metric because a missed escalation (false negative) carries a higher business cost than a false alarm (false positive).

Precision: The fraction of flagged tickets that genuinely miss SLA. At 61.5%, roughly 62 of every 100 manager reviews involve a ticket that truly needed attention. The remaining 38 are false alarms. Precision determines the operational workload the model creates. If precision falls too low, alert fatigue causes managers to stop trusting the flags.

F1 Score: The harmonic mean of Precision and Recall. It rewards models that are simultaneously selective and sensitive. Your F1 of 0.696 confirms the model is not gaming Recall by flagging everything — it remains precise enough to be operationally useful. The near-unchanged F1 when moving from threshold 0.50 to 0.465 (0.696 → 0.696) confirms the threshold adjustment was efficient: you gained 3.9 Recall points while losing only 2.1 Precision points.

ROC-AUC: If you randomly draw one ticket that missed SLA and one that met it, the model assigns a higher risk score to the risky one 83.9% of the time. This measures the quality of the probability scores independently of any threshold. An improvement from 0.803 (LR) to 0.839 (XGBoost) means XGBoost's sorted risk queue places genuinely risky tickets higher — managers reviewing the top 20% of flagged tickets see a higher concentration of true breaches with XGBoost than with LR.

PR-AUC: Precision-Recall AUC is more informative than ROC-AUC for imbalanced problems. Your PR-AUC of 0.760 against a random baseline of ~0.366 (the miss rate itself) confirms the model provides substantial lift specifically on the events that matter — the breaches.

The Threshold as a Business Dial

The threshold of 0.465 is not a fixed property of the model — it is an operational decision that management can adjust based on team capacity and breach cost tolerance.

- **Raise the threshold toward 0.60–0.70:** Fewer tickets flagged. Precision rises — a higher fraction of flagged tickets are genuine. Recall falls — more breaches slip through. Use this when manager capacity is limited and false alarms are generating alert fatigue.

- **Lower the threshold toward 0.30–0.40:** More tickets flagged. Recall rises — fewer breaches go undetected. Precision falls — more false alarm reviews. Use this when SLA breach costs are high — financial penalties, major clients — and the team can absorb the extra review workload.
- **At 0.465 (recommended):** 47.6% of tickets flagged. Recall 80.1%. Precision 61.5%. At 24,918 annual incidents, this means approximately 11,861 tickets flagged per year (~990 per month). Of those, ~7,292 are genuine catches and ~4,569 are false alarms requiring a brief review.

Business framing for leadership: The threshold question should be put to operations management as a staffing question: 'Can your team leads review approximately 990 flagged tickets per month? If yes, deploy at 0.465. If the team can only handle 700 reviews per month, raise the threshold to 0.50. If 1,200 reviews are feasible, lower it to 0.40.' The model provides the mechanism; the business decides the operating point.

Threshold	Precision	Recall	F1	% Tickets Flagged
0.50 (default)	0.636	0.762	0.693	~35%
0.465 (tuned) ★	0.615	0.801	0.696	~48%
0.40	0.582	0.860	0.694	~54%
0.30	0.539	0.900	0.674	~61%

Confusion Matrix — XGBoost at Tuned Threshold (0.465)

	Predicted: Met SLA	Predicted: Missed SLA
Actual: Met SLA	2,248 — True Negatives Correctly identified as safe. No action needed.	913 — False Positives Flagged unnecessarily. Minor cost: unneeded review.
Actual: Missed SLA	363 — False Negatives Missed escalations. Most costly outcome — breach occurs with no warning.	1,460 — True Positives Correctly flagged for intervention. These are the manager saves.

Confusion Matrix — Business Meaning of Each Cell

- **True Negatives — 2,248 tickets:** Tickets correctly identified as safe. They met SLA and received no unnecessary manager attention. Scaled to annual operations: ~11,200 tickets correctly left alone per year. This is the model operating invisibly and correctly in the background.
- **False Positives — 913 tickets:** Tickets flagged as risky that actually met SLA — false alarms. Each requires a brief manager review (estimated 5–10 minutes). At 5 minutes per review: $913 \times 5 = 4,565$ minutes (~76 hours, ~10 working days) per test period spent on unnecessary reviews. Annually: ~4,565 false alarm reviews per year. This is the operational cost of achieving 80% Recall and is the price paid for catching the true breaches.
- **False Negatives — 363 tickets:** Tickets that missed SLA but were not flagged. These are the model's undetected failures — breaches that occurred with no warning. Scaled annually: ~1,815 breaches per year go undetected. At the default threshold of 0.50 this was 434; threshold tuning to 0.465 reduced it to 363, preventing an additional 71 undetected breaches per test period (~355 per year).
- **True Positives — 1,460 tickets:** Tickets correctly flagged for proactive intervention. These are the 'manager saves' — tickets where early warning created the opportunity to escalate, reassign, or expedite. Annually: ~7,300 correct flags per year. Even if intervention successfully prevents breach in only 30% of cases, the model prevents approximately 2,190 SLA breaches per year.

6D. Error Analysis

Understanding where the model fails is as important as reporting where it succeeds.

Business implication of false negatives by priority: Priority 2 (High) tickets miss SLA at 97.1%, meaning nearly every one is a positive case. The model learns this and flags most of them. The harder false negatives are Moderate-priority tickets (Priority 3) — which make up 95% of the dataset — that fail for subtle reasons not encoded in the 18 open-time features. The specific technician assigned, the actual complexity of the underlying issue, and the resolution path taken are all invisible to the model at ticket creation. These are the breaches the model cannot prevent — they require human judgment and experience to identify.

Deployment risk from false positives: At threshold 0.465, approximately 38.5% of flagged tickets are false alarms. If managers repeatedly see false alarms from specific categories — such as Category 45 and Category 40, where high base miss rates cause the model to flag most incoming tickets — they may begin ignoring the model's alerts for those categories entirely. This behavioral response would undermine Recall through manager disengagement rather than model failure. Monitoring false-positive rate by category weekly is therefore an essential post-deployment metric, with a protocol to review threshold or retraining if the FP rate exceeds 60% for any category for four or more consecutive weeks.

False Negatives by Priority (Missed Escalations)

Priority 2 (High) tickets miss SLA at 97.1% in raw data, meaning nearly every one is a positive case. At threshold 0.465, the model's 363 false negatives are disproportionately concentrated in Moderate-priority tickets (Priority 3), which constitute 95% of the dataset. The model has learned that Priority 2 tickets are almost always risky and flags most of them; the harder cases are Moderate-priority tickets that fail for subtle reasons not captured by the 18 open-time features — such as the specific technician assigned or the complexity of the underlying issue, neither of which is available at open time.

False Positives and Operational Cost

At threshold 0.465, 913 tickets are flagged as risky but ultimately meet SLA (false positives). These generate unnecessary manager reviews. The precision of 61.5% means roughly 38.5 cents of every review dollar spent on flagged tickets is wasted on safe tickets. This is the direct operational cost of the chosen threshold. Raising the threshold to 0.50 reduces false positives to 795 but increases false negatives from 363 to 434.

Failure Modes Paragraph

The model's primary failure mode is missing Moderate-priority tickets (Priority 3) that have unusual breach patterns not visible in open-time fields. These tickets look ordinary when filed — common category, standard priority, assigned group — but fail for reasons that emerge during ticket execution. A secondary failure mode is over-flagging tickets in high-miss-rate categories such as Category 45 and Category 40, where the model correctly learns high base rates but cannot distinguish individual tickets that will beat those rates. The deployment risk is that managers who repeatedly see false alarms from a specific category may begin ignoring model alerts for that category — undermining Recall through behavioral response rather than model error. Monitoring the false-positive rate by category is therefore an essential post-deployment metric.

7. Decision Insight & Implementation Plan

7.1 Monday Morning Actions

- **Week 1:** Map anonymised field values (Group 9, Category 45, etc.) back to real operational names using the ITSM platform's reference tables. Without this mapping, the model's outputs cannot be acted on.
- **Week 2:** Score all open tickets in the current queue using the trained XGBoost model at threshold 0.465. Identify tickets in the top-risk group that have been open for more than 48 hours without resolution.
- **Week 3:** Present findings to Group 9, Group 10, and Group 31 team leads. Review SLA window targets for Priority 1 and Priority 2 with service management leadership — the 97.1% miss rate for High-priority tickets indicates the targets are unachievable under current capacity.
- **Ongoing:** Integrate the model into the ITSM ticketing platform so new tickets are scored at creation and risk flags appear automatically in the assignment queue.

7.2 Operational Plan

Who uses it: Service desk team leads and managers. Technicians do not need to see scores directly — the flag appears as a queue priority indicator.

When: At ticket creation. The model scores a ticket within milliseconds of the opened_at timestamp using 18 fields available from the first event row.

How: The sklearn Pipeline (preprocessor + XGBoost) is serialized with joblib. A scoring API endpoint accepts a ticket payload and returns a probability score and binary flag. The flag feeds into the ITSM dashboard queue view.

7.3 Cost/Benefit Analysis

At 80% Recall on 24,918 annual incidents (36.6% miss rate = 9,115 at-risk tickets per year), the model correctly flags approximately 7,292 at-risk tickets annually. If intervention prevents breach in 30% of those cases, roughly 2,188 SLA breaches are avoided per year. For enterprise organizations with SLA breach costs between \$500–\$5,000 per incident (contract penalties, escalation labor, customer impact), this represents an estimated \$1.1M–\$11M annual value. Model operational cost: infrastructure for the scoring API, quarterly retraining runs, and monitoring dashboards — estimated at \$20,000–\$50,000 per year depending on team size.

Detailed cost breakdown: False alarm review cost: 4,565 reviews/year × 7 min average = 533 hours of team lead time per year (~67 working days). At a fully-loaded team lead salary of \$80,000/year (\$40/hr), this is approximately \$21,300/year in review time. Against this, consider that each prevented SLA breach avoids escalation labor (estimated 2–4 hours of senior management time at \$60–\$100/hr), contractual penalties (variable by client, commonly \$500–\$5,000 per incident), and reputational cost. At a conservative \$500 per breach and 2,190 prevented breaches, the model generates ~\$1.1M in annual value. At \$5,000 per breach, the value rises to ~\$11M. Net of the ~\$21,300 in false-alarm review costs and \$20,000–\$50,000 in infrastructure and retraining, the model produces a minimum estimated ROI of 20× at conservative breach cost estimates.

7.4 Monitoring Plan

Recommendation	Benefit	Cost / Risk	How to Monitor
Deploy at threshold 0.465	80% Recall; catches 4 in 5 at-risk tickets	48% of tickets flagged; ~38% are false alarms	Track live Recall monthly vs 75% floor trigger
Escalate Groups 9, 10, 31 immediately	Targets highest-breach groups; highest ROI per review	May create perception of group targeting; requires HR sensitivity	Monthly miss rate per group; alert if any group exceeds 90%
Retrain quarterly	Keeps model calibrated to current ticket patterns	Requires labeled data pipeline and retraining budget	ROC-AUC on rolling 90-day test window; drift in feature distributions
Monitor false-positive rate by category	Prevents alert fatigue in high-miss-rate categories	Adds monitoring overhead	Weekly FP rate per category; flag if FP rate > 60% in any category for 4+ weeks
Track null rates and row counts in live data feed	Detects data pipeline failures before they corrupt model scores	Monitoring infrastructure required	Daily pipeline health check: row count within $\pm 20\%$ of prior 7-day average; null rate per feature

7.5 Ethics & Risk Note

- **Fairness:** Field values are anonymised, making direct fairness analysis (e.g., by caller demographics) impossible. In production, teams should audit whether model flag rates differ by caller ID groups that correlate with protected characteristics.
- **Misuse risk:** The model identifies high-risk assignment groups. If used to evaluate individual technician performance rather than systemic capacity issues, it could unfairly penalize teams handling inherently harder ticket types.
- **Human oversight:** The model should be positioned as a decision support tool, not an automated escalation trigger. A human team lead should confirm every escalation decision.
- **Privacy:** The model uses caller_id and opened_by in a grouped form. Individual-level scores should not be surfaced to external parties.

8. Limitations

8.1 Data Limitations

- **Single-organization scope:** The dataset covers one anonymous IT organization. SLA window definitions, ticket categorization schemes, and team structures vary significantly across organizations. Model performance on a different organization's data is unknown without retraining.
- **Anonymisation:** All category, group, location, and symptom values are anonymised (e.g., 'Category 45', 'Group 9'). Operational interpretation and deployment require mapping these back to real names, which is not possible from the dataset alone.
- **Earliest-row approximation:** Predictors are taken from the earliest available event row per incident. This approximates but does not guarantee the true ticket-creation state. For a subset of incidents, the earliest captured snapshot may already reflect a later lifecycle state.
- **made_sla instability:** 36.6% of incidents have varying made_sla across rows. The final-row value is used as the definitive outcome — defensible but derived, not directly observed at a single point in time.
- **Fixed time window:** Data covers Feb 2016 – Feb 2017. Organizational changes, SLA policy revisions, or staffing changes after this window are not reflected. The model should be retrained on more recent data before deployment.

8.2 Modeling Limitations

- **No hyperparameter tuning:** RF and XGBoost hyperparameters were set manually based on common defaults. Systematic tuning with GridSearchCV or RandomizedSearchCV would likely improve performance.
- **No temporal validation:** The 80/20 split is random rather than time-ordered. A temporal split (train on earlier months, test on later months) would give a more rigorous estimate of real-world performance and model stability over time.
- **No explainability:** Feature importance scores (e.g., via SHAP) are not included. Managers cannot currently understand why a specific ticket was flagged — which limits trust and actionability.
- **Borderline fields not fully resolved:** reassignment_count and reopen_count produced zero performance gain but may become available at ticket creation in some ITSM configurations. Their exclusion is conservative and appropriate but worth revisiting in a production context.
- **Calibration tradeoff:** The selected XGBoost model is not probability-calibrated. If the deployment system needs trustworthy probability estimates (e.g., to display '73% risk' to managers), the calibrated version should be used despite its lower Recall at default threshold.

9. Appendix

Appendix A — Full Data Dictionary (36 Raw Fields)

Field	Type	Example Value	Miss %	Description
number	String	INC0000045	0%	Unique incident identifier — grain key
incident_state	Categorical	New, Resolved, Closed	0%	Current workflow state — used for ETL sorting only; excluded from model (leakage)
active	Boolean	True / False	0%	Whether ticket is active — excluded (leakage)
reassignment_count	Integer	0–21	0%	Times ticket reassigned — borderline leakage; excluded from baseline model
reopen_count	Integer	0–6	0%	Times ticket reopened — borderline leakage; excluded from baseline model
sys_mod_count	Integer	0–101	0%	Record modification count — post-open, excluded (leakage)
made_sla	Boolean	True / False	0%	Target variable — SLA met (True) or missed (False). Inverted to miss_sla = 1 if False
caller_id	String	Caller 2403	0%	Anonymised caller identifier — not used in model (too many levels)
opened_by	String	Opened by 8	2.9%	Who created the ticket — not used in model
opened_at	Datetime	29/2/2016 01:16	0%	Ticket creation timestamp — used to derive time features
sys_created_by	String	Created by 6	37.5%	System creator — excluded (high missingness, unclear value)
sys_created_at	Datetime	29/2/2016 01:23	0%	System creation timestamp — not used in model
sys_updated_by	String	Updated by 21	0%	Used for ETL sort order only
sys_updated_at	Datetime	29/2/2016 01:23	0%	Primary ETL sort key
contact_type	Categorical	Phone, Email, Self-service	~1%	How ticket was submitted — MODEL FEATURE
location	Categorical	Location 143	0.1%	Ticket location — MODEL FEATURE (top-10+Other)
category	Categorical	Category 55	0%	Service category (55 values) — MODEL FEATURE (top-15+Other)
subcategory	Categorical	Subcategory 170	0%	Service subcategory (246 values) — MODEL FEATURE (top-20+Other)

u_symptom	Categorical	Symptom 72	23.2%	Reported symptom (~100 values) — MODEL FEATURE (top-15+Other+Unknown)
cmdb_ci	String	?	99.7%	Config item — EXCLUDED (≥98% missing)
impact	Ordinal	2 - Medium	0%	Business impact level — MODEL FEATURE (ordinal encoded 1–3)
urgency	Ordinal	2 - Medium	0%	Urgency level — MODEL FEATURE (ordinal encoded 1–3)
priority	Ordinal	3 - Moderate	0%	Computed priority — MODEL FEATURE (ordinal encoded 1–4)
assignment_group	Categorical	Group 56	10.0%	Handling group — MODEL FEATURE (top-20+Other; Unknown for missing)
assigned_to	String	Resolver 89	19.4%	Individual assignee — EXCLUDED (too many levels; use group instead)
knowledge	Boolean	True / False	0%	Knowledge article used — MODEL FEATURE (cast to 0/1)
u_priority_confirmation	Boolean	True / False	0%	Priority confirmed by manager — not used in model
notify	Categorical	Do Not Notify	0%	Notification preference — MODEL FEATURE
problem_id	String	?	98.4%	Related problem — EXCLUDED (≥98% missing)
rfc	String	?	99.3%	Change request — EXCLUDED (≥98% missing)
vendor	String	?	99.8%	Vendor involved — EXCLUDED (≥98% missing)
caused_by	String	?	100%	Cause — EXCLUDED (100% missing)
closed_code	Categorical	code 5	0%	Resolution code — EXCLUDED (leakage: not known at open time)
resolved_by	String	Resolved by 149	0%	Resolver ID — EXCLUDED (leakage)
resolved_at	Datetime	29/2/2016 11:29	0%	Resolution timestamp — EXCLUDED (leakage)
closed_at	Datetime	5/3/2016 12:00	~6%	Close timestamp — EXCLUDED (leakage; used only for target derivation)

Appendix B — Engineered Features Reference

Feature Name	Derived From	Values	Rationale
impact_num	impact	1, 2, 3	Numeric encoding of ordinal field

urgency_num	urgency	1, 2, 3	Numeric encoding of ordinal field
priority_num	priority	1, 2, 3, 4	Numeric encoding of ordinal field
has_symptom	u_symptom	0 / 1	1 if symptom recorded at open time; absence may indicate vague issue
has_assignment	assignment_group	0 / 1	1 if group assigned at open time; unassigned tickets may idle longer
is_phone_contact	contact_type	0 / 1	1 if submitted by phone; phone tickets often more complex
priority_urgency_match	priority_num, urgency_num	0 / 1	1 if priority == urgency; mismatch may indicate mis-classification at creation
opened_month	opened_at	1–12	Captures seasonal patterns in ticket volume and staffing
opened_dayofweek	opened_at	0–6 (Mon–Sun)	Weekend/Friday tickets may have longer resolution times
opened_hour	opened_at	0–23	After-hours tickets may not be picked up until next business day

Appendix C — Leakage Sensitivity Test Results

Adding reassignment_count and reopen_count to Logistic Regression produced zero improvement across all metrics. This confirms that safe open-time features are sufficient and borderline fields are excluded without performance cost.

Metric	LR — Safe Only	LR — + Borderline Fields
Accuracy	0.7253	0.7253
Precision	0.6105	0.6105
Recall	0.6879	0.6879
F1	0.6469	0.6469
ROC-AUC	0.8028	0.8028
PR-AUC	0.6955	0.6955

Decision: Borderline fields excluded. Zero improvement + leakage risk = no justification for inclusion.

Appendix D — Threshold Tuning Full Table

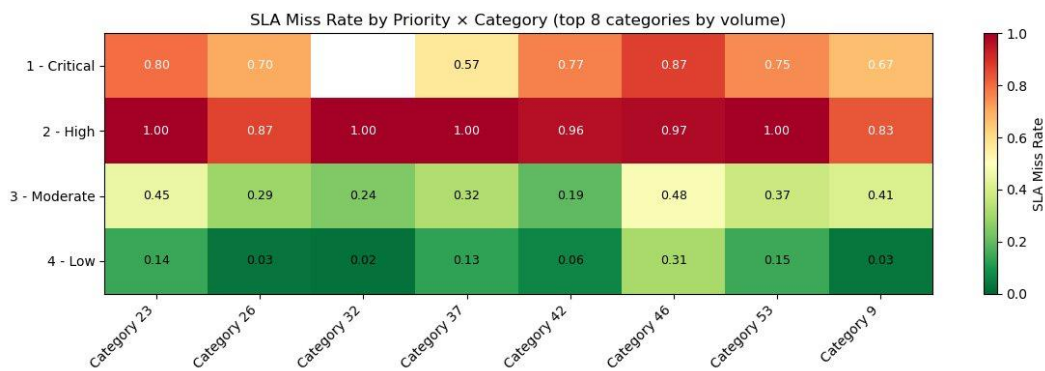


Chart 8 — SLA Miss Rate by Priority × Category (Top 8 Categories by Volume)

Threshold	Precision	Recall	F1	% Flagged	Notes
0.10	0.419	0.992	0.590	86.5%	Flags nearly everything
0.20	0.480	0.949	0.638	72.3%	
0.30	0.539	0.900	0.674	61.1%	
0.40	0.582	0.860	0.694	54.0%	
0.465 ★	0.615	0.801	0.696	47.6%	Selected threshold — meets 80% Recall target
0.50 (default)	0.636	0.762	0.693	35.0%	Default operating point
0.60	0.668	0.682	0.675	29.9%	
0.70	0.716	0.557	0.626	22.7%	
0.80	0.770	0.384	0.513	14.5%	Very conservative; misses many